Computación y estructuras discretas I

Unidad 3: Estructuras Discretas no recursivas y análisis de algoritmos — Complejidad Espacial

Juan Marcos Caicedo Mejía – jmcaicedo@icesi.edu.co

Departamento de Computación y Sistemas Inteligentes
Facultad Barberi de Ingeniería, Diseño y Ciencias Aplicadas
Universidad ICESI
Material adaptado del material original del profesor Oscar Bedoya

2026

Tabla de contenido

1. ¿Qué es la Complejidad Espacial?
2. Componentes de la Complejidad Espacial
3. Tipos de Complejidad Espacial
4. Análisis de Espacio en Algoritmos Iterativos
5. Análisis de Espacio en Algoritmos Recursivos
6. Comparación Tiempo vs. Espacio
7. Ejercicios y Consulta

Contenido

1. ¿Qué es la Complejidad Espacial?

2. Componentes de la Complejidad Espacial

3. Tipos de Complejidad Espacial

4. Análisis de Espacio en Algoritmos Iterativos

5. Análisis de Espacio en Algoritmos Recursivos

6. Comparación Tiempo vs. Espacio

7. Ejercicios y Consulta

Complejidad Espacial

Motivación

Hasta ahora hemos analizado **cuánto tiempo** tarda un algoritmo en función del tamaño de la entrada.

Pero también debemos preguntarnos: **¿cuánta memoria consume?**

- En dispositivos con poca RAM (sistemas embebidos, móviles) el espacio es un recurso crítico.
- Un algoritmo *rápido* pero que consume demasiada memoria puede ser inutilizable en la práctica.
- Al diseñar algoritmos debemos equilibrar **tiempo** y **espacio**.

Complejidad Espacial

Definición formal

La **complejidad espacial** de un algoritmo es la cantidad **total** de memoria que necesita en función del tamaño de la entrada n.

Se compone de dos partes:

$$S(n) = \underbrace{\text{Espacio de entrada}}_{\text{input space}} + \underbrace{\text{Espacio auxiliar}}_{\text{auxiliary space}}$$

- **Espacio de entrada:** memoria ocupada por los datos que recibe el algoritmo (los parámetros).
- **Espacio auxiliar:** memoria *extra* que el algoritmo utiliza internamente (variables temporales, estructuras intermedias, pila de llamadas).

Complejidad Espacial

Espacio auxiliar vs. complejidad espacial

Confusión frecuente

Muchas fuentes usan "complejidad espacial" cuando en realidad hablan de "espacio auxiliar".

Ejemplo con algoritmos de ordenamiento sobre un arreglo de n elementos:

Algoritmo	**Espacio auxiliar**	**Complejidad espacial**
Insertion Sort	$O(1)$	$O(n)$
Heap Sort	$O(1)$	$O(n)$
Merge Sort	$O(n)$	$O(n)$

La *complejidad espacial total* incluye siempre el arreglo de entrada ($O(n)$), por eso todos resultan $O(n)$. Lo que diferencia a Merge Sort es su **espacio auxiliar**.

Contenido

1. ¿Qué es la Complejidad Espacial?

2. Componentes de la Complejidad Espacial

3. Tipos de Complejidad Espacial

4. Análisis de Espacio en Algoritmos Iterativos

5. Análisis de Espacio en Algoritmos Recursivos

6. Comparación Tiempo vs. Espacio

7. Ejercicios y Consulta

Complejidad Espacial

¿Qué ocupa espacio en un algoritmo?

- **Variables simples** (enteros, booleanos, flotantes): espacio constante $O(1)$ cada una.
- **Arreglos / listas** de tamaño n: espacio $O(n)$.
- **Matrices** de $n \times n$: espacio $O(n^2)$.
- **Pila de llamadas recursivas**: cada llamada agrega un *frame* a la pila; la profundidad de recursión determina el espacio.

Al igual que con el tiempo, usamos la **notación** O para expresar el crecimiento del espacio en el **peor caso**.

Complejidad Espacial

Espacio de la pila de llamadas (call stack)

Cada vez que una función llama a otra (o a sí misma), se reserva un **frame** en la pila con:

- La dirección de retorno.
- Los parámetros de la función.
- Las variables locales.

Cuando la función termina, el frame se libera. Lo importante es el **máximo número de frames simultáneos**.

Clave

El espacio de la pila depende de la **profundidad máxima** de recursión, no del número total de llamadas.

Contenido

1. ¿Qué es la Complejidad Espacial?

2. Componentes de la Complejidad Espacial

3. Tipos de Complejidad Espacial

4. Análisis de Espacio en Algoritmos Iterativos

5. Análisis de Espacio en Algoritmos Recursivos

6. Comparación Tiempo vs. Espacio

7. Ejercicios y Consulta

Complejidad Espacial

Complejidad $O(1)$ — Espacio constante

El algoritmo usa una cantidad **fija** de memoria sin importar el tamaño de la entrada.

Ejemplo: suma de un arreglo

```
sum(arr, n):
  total <- 0          // 1 variable
  i <- 1              // 1 variable
  while i <= n:
    total <- total + arr[i]
    i <- i + 1
  return total
```

Análisis:

- `total` e `i`: 2 variables de tamaño fijo.
- No se crean estructuras adicionales.
- El arreglo `arr` es la *entrada*, no cuenta como auxiliar.

$$S_{\text{aux}}(n) = O(1)$$

Complejidad Espacial

Complejidad $O(n)$ — Espacio lineal

La memoria crece **proporcionalmente** al tamaño de la entrada.

Ejemplo: copiar un arreglo

```
copy(arr, n):
  result <- nuevo arreglo[n]
  i <- 1
  while i <= n:
    result[i] <- arr[i]
    i <- i + 1
  return result
```

Análisis:

- `result`: arreglo de n elementos $\Rightarrow O(n)$.
- `i`: variable simple $\Rightarrow O(1)$.
- Total auxiliar: $O(n) + O(1) = O(n)$.

$$S_{\text{aux}}(n) = O(n)$$

Complejidad Espacial

Complejidad $O(n^2)$ — Espacio cuadrático

Ocurre cuando se crea una **estructura bidimensional** de tamaño $n \times n$.

Ejemplo: crear una tabla de distancias

```
initMatrix(n):
  M <- nueva matriz[n][n]
  i <- 1
  while i <= n:
    j <- 1
    while j <= n:
      M[i][j] <- 0
      j <- j + 1
    i <- i + 1
  return M
```

Análisis:

- `M`: matriz de n^2 elementos $\Rightarrow O(n^2)$.
- `i`, `j`: variables simples $\Rightarrow O(1)$.

$$S_{\text{aux}}(n) = O(n^2)$$

Impractical para n grande (ej. $n = 10\,000 \Rightarrow 10^8$ celdas).

Complejidad Espacial

Resumen de tipos de complejidad espacial

Notación	Nombre	Ejemplo típico
$O(1)$	Constante	Variables simples, intercambio in-place
$O(\log n)$	Logarítmica	Búsqueda binaria recursiva (pila)
$O(n)$	Lineal	Copiar arreglo, recursión lineal
$O(n^2)$	Cuadrática	Matriz $n \times n$, tabla de adyacencia

Contenido

1. ¿Qué es la Complejidad Espacial?

2. Componentes de la Complejidad Espacial

3. Tipos de Complejidad Espacial

4. Análisis de Espacio en Algoritmos Iterativos

5. Análisis de Espacio en Algoritmos Recursivos

6. Comparación Tiempo vs. Espacio

7. Ejercicios y Consulta

Complejidad Espacial

Método: análisis línea a línea

Para calcular el espacio auxiliar de un algoritmo iterativo:

1. Identificar **todas** las variables y estructuras declaradas.
2. Determinar si su tamaño **depende de** n o es constante.
3. Sumar los aportes de cada componente.
4. Expresar el resultado con la cota asintótica más ajustada.

Regla práctica

Variables simples (`int`, `bool`, `float`) $\Rightarrow O(1)$.
Arreglos / estructuras de tamaño $n \Rightarrow O(n)$.
Matrices $n \times n \Rightarrow O(n^2)$.

Complejidad Espacial

Ejemplo iterativo: suma de matrices

#	Instrucción	Espacio
1	$i \leftarrow 1$	$O(1)$
2	$j \leftarrow 1$	$O(1)$
3	$mat3 \leftarrow$ nueva matriz $[n][n]$	$O(n^2)$
4	**while** $i \leq n$	–
5	**while** $j \leq n$	–
6	$mat3[i][j] \leftarrow mat1[i][j] + mat2[i][j]$	–
7	$j \leftarrow j + 1$	–
8	$i \leftarrow i + 1$	–

Entrada: $mat1, mat2$ de $n \times n \Rightarrow$ no cuentan como auxiliar.

Auxiliar:

- $mat3$: nueva matriz $n \times n \Rightarrow O(n^2)$
- i, j: enteros $\Rightarrow O(1)$

$$S_{aux}(n) = O(n^2) + O(1) = O(n^2)$$

El término dominante es la matriz resultado.

Complejidad Espacial

Ejemplo iterativo: búsqueda lineal

#	Instrucción	Espacio
1	$i \leftarrow 1$	$O(1)$
2	**while** $i \leq n$	—
3	**if** $arr[i] = target$	—
4	**return** i	—
5	$i \leftarrow i + 1$	—
6	**return** -1	—

Entrada: *arr* de *n* elementos, *target* $\Rightarrow$ no cuentan como auxiliar.

Auxiliar:

- *i*: un entero $\Rightarrow$ $O(1)$
- No se crea ninguna estructura adicional.

$$S_{\text{aux}}(n) = O(1)$$

Independientemente del tamaño del arreglo, solo se usa una variable extra.

Contenido

1. ¿Qué es la Complejidad Espacial?

2. Componentes de la Complejidad Espacial

3. Tipos de Complejidad Espacial

4. Análisis de Espacio en Algoritmos Iterativos

5. Análisis de Espacio en Algoritmos Recursivos

6. Comparación Tiempo vs. Espacio

7. Ejercicios y Consulta

Complejidad Espacial

Recursión y la pila de llamadas

En los algoritmos recursivos, **cada llamada activa** agrega un frame a la pila.

Lo relevante para el espacio es la **profundidad máxima** de la pila en cualquier momento de la ejecución.

Fórmula

$$S_{\text{pila}}(n) = \text{profundidad máxima de recursión} \times O(1) \text{ por frame}$$

Importante

Tener muchas llamadas en total **no implica** que todas estén activas al mismo tiempo.

Complejidad Espacial

Ejemplo recursivo: suma acumulada

```
add(n):
  if n <= 0:
    return 0
  return n + add(n - 1)
```

Traza para $n = 4$:

- `add(4)` llama a `add(3)`
- `add(3)` llama a `add(2)`
- `add(2)` llama a `add(1)`
- `add(1)` llama a `add(0)`
- `add(0)` retorna 0 $\longleftarrow$ base

Análisis:

La pila tiene **simultáneamente** los frames de `add(4)`, `add(3)`, ..., `add(0)`.

Profundidad máxima $= n + 1 \approx n$.

$$S_{\text{aux}}(n) = O(n)$$

Aunque el número de llamadas totales también es n, aquí coincide porque son *todas* simultáneas.

Complejidad Espacial

Cuidado: no todas las llamadas son simultáneas

```
addSequence(n):
  sum <- 0
  i <- 1
  while i < n:
    sum <- sum + pairSum(i, i+1)
    i <- i + 1
  return sum

pairSum(x, y):
  return x + y
```

Análisis:

`pairSum` es llamada $\approx n$ veces en total, pero **de una en una**: cuando termina, su frame se libera antes de la siguiente llamada.

En cualquier momento solo hay **1 frame** de `pairSum` en la pila.

$$S_{\text{aux}}(n) = O(1)$$

Conclusión clave

El espacio de la pila depende de las llamadas **activas simultáneamente**, no del total de llamadas realizadas.

Complejidad Espacial

Ejemplo recursivo: factorial

```
factorial(n):
  if n <= 1:
    return 1
  return n * factorial(n - 1)
```

Traza para $n = 5$:

$$5! \to 4! \to 3! \to 2! \to 1!$$

Pila activa simultáneamente:

$$\underbrace{f(5), f(4), f(3), f(2), f(1)}_{n \text{ frames}}$$

Análisis del espacio:

- Cada frame ocupa espacio para el parámetro n y la dirección de retorno: $O(1)$ por frame.
- La profundidad máxima de la pila es n.

$$S_{\text{aux}}(n) = O(n)$$

Contraste: una versión *iterativa* del factorial solo usa $O(1)$ de espacio auxiliar.

Contenido

1. ¿Qué es la Complejidad Espacial?

2. Componentes de la Complejidad Espacial

3. Tipos de Complejidad Espacial

4. Análisis de Espacio en Algoritmos Iterativos

5. Análisis de Espacio en Algoritmos Recursivos

6. Comparación Tiempo vs. Espacio

7. Ejercicios y Consulta

Complejidad Espacial

Relación entre tiempo y espacio

El tiempo y el espacio son dos **recursos independientes** que deben equilibrarse según el contexto.

Algoritmo	Tiempo	Espacio auxiliar
Búsqueda lineal	$O(n)$	$O(1)$
Búsqueda binaria iterativa	$O(\log n)$	$O(1)$
Búsqueda binaria recursiva	$O(\log n)$	$O(\log n)$
Merge Sort	$O(n \log n)$	$O(n)$
Insertion Sort	$O(n^2)$	$O(1)$

Merge Sort es más *rápido* que Insertion Sort para entradas grandes, pero requiere más *espacio*.

Complejidad Espacial

La disyuntiva espacio–tiempo

Ganar tiempo a costa de espacio

- Guardar resultados pre-calculados en una tabla (memoización).
- Usar una tabla hash para búsquedas $O(1)$ en vez de $O(n)$.

Ganar espacio a costa de tiempo

- Reemplazar recursión por iteración.
- Recalcular valores en lugar de almacenarlos.
- Ordenar in-place (Heap Sort, Insertion Sort).

Principio general

No existe una opción universalmente mejor: la elección depende de los **recursos disponibles** y los **requisitos del sistema**.

Contenido

1. ¿Qué es la Complejidad Espacial?

2. Componentes de la Complejidad Espacial

3. Tipos de Complejidad Espacial

4. Análisis de Espacio en Algoritmos Iterativos

5. Análisis de Espacio en Algoritmos Recursivos

6. Comparación Tiempo vs. Espacio

7. Ejercicios y Consulta

Complejidad Espacial

Ejercicios

Determine la complejidad espacial auxiliar de los siguientes algoritmos:

- Un algoritmo que invierte un arreglo **en sitio** (sin crear uno nuevo), usando solo una variable temporal.
- Un algoritmo que crea una copia de un arreglo de n enteros y luego la ordena.
- La función recursiva `fibonacci(n)` que calcula el n-ésimo número de Fibonacci retornando `fibonacci(n-1) + fibonacci(n-2)`.
- Búsqueda binaria implementada de forma **iterativa** sobre un arreglo ordenado.

Complejidad Espacial

Ejercicios (continuación)

Para el siguiente pseudocódigo, calcule la complejidad espacial auxiliar:

```
buildTable(n):
  tabla <- nuevo arreglo[n]
  i <- 1
  while i <= n:
    tabla[i] <- i * i
    i <- i + 1
  return tabla
```

¿Cambia el resultado si `tabla` se declara **fuera** de la función y se pasa como parámetro?

Consulta

- ¿Por qué Merge Sort tiene espacio auxiliar $O(n)$ mientras que Heap Sort tiene $O(1)$? Explique con base en lo que hace cada algoritmo internamente.
- ¿En qué situaciones práctica es preferible un algoritmo con mayor complejidad espacial pero menor complejidad temporal?